

数据结构实验报告

软件学院 软件工程二班 李宗杭 2311451

实验题目：表达式求值

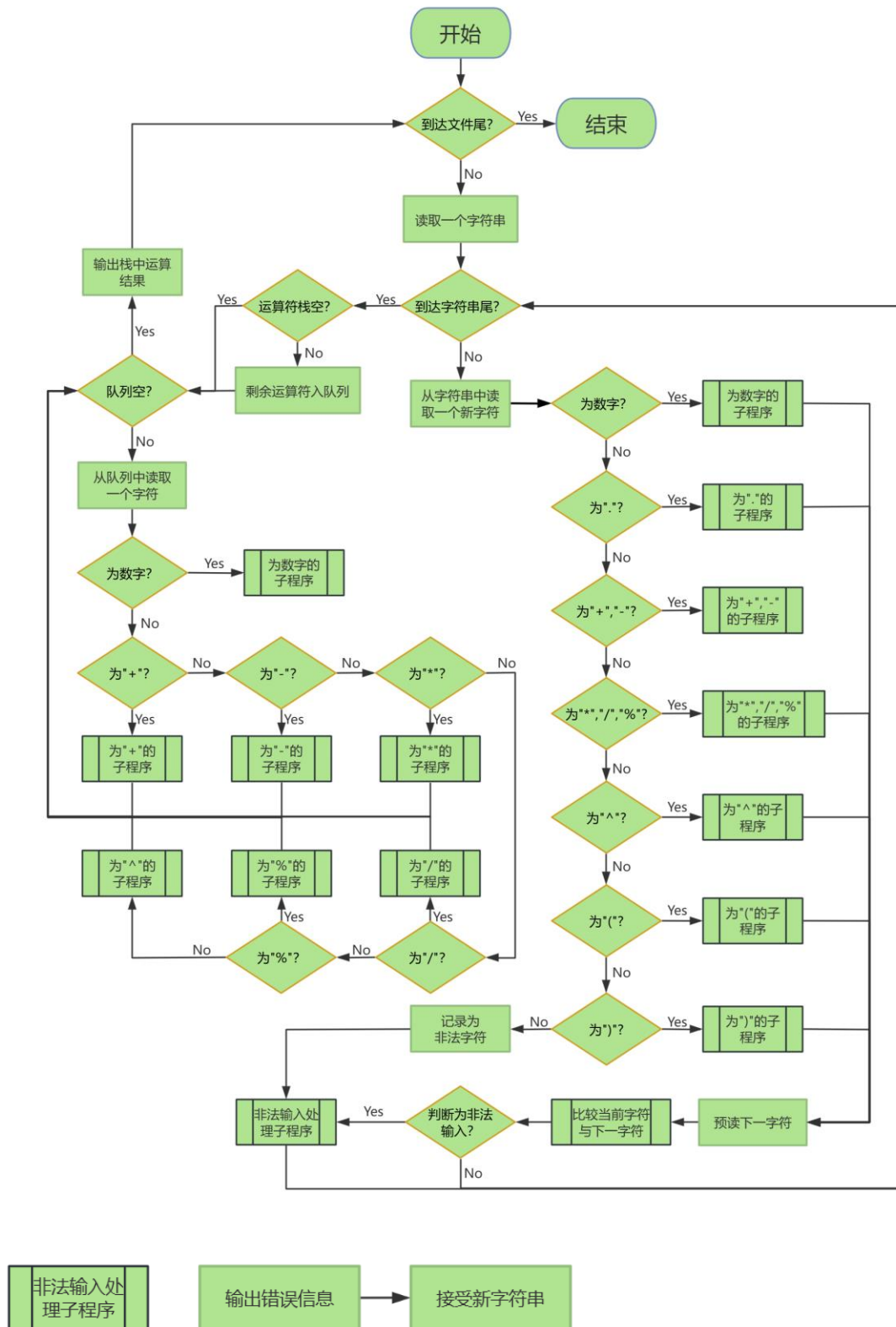
一、算法说明：

该算法采用先将中缀表达式转为后缀表达式，再利用后缀表达式进行计算的方法。

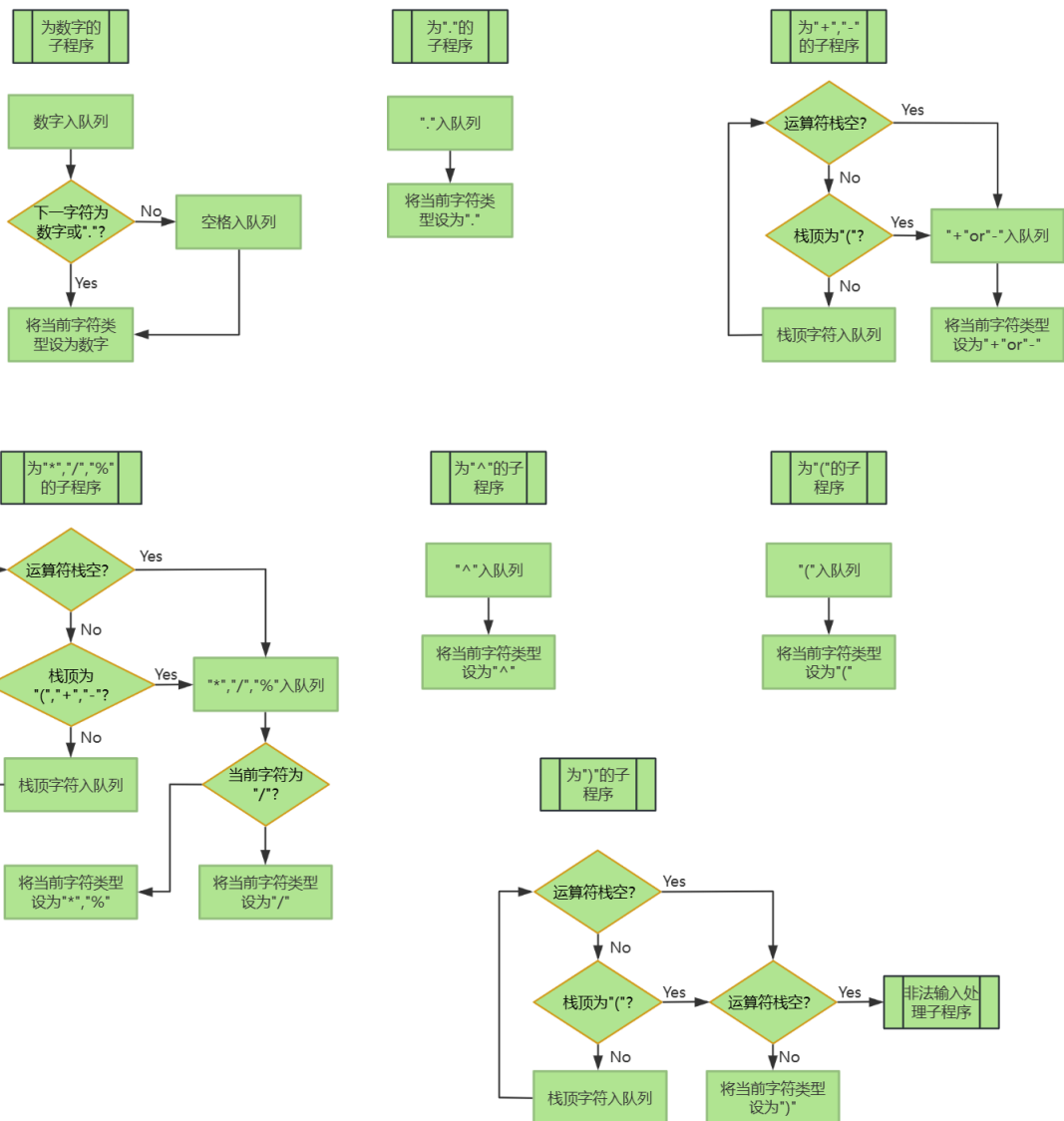
- 1、首先创建三个数据结构：一个 double 类型栈用于后缀计算存储操作数；一个 char 类型栈用于中缀转后缀存储运算符；一个 char 队列用于存储后缀表达式。
- 2、中缀转后缀：使用队列和 char 栈。循环从中缀表达式字符串中逐字符读取，并预判下一字符。使用数字来记录当前与下一字符的类型，在单次循环结束时判断是否有非法输入。遇到“0~9”或“.”直接入队列；遇到“(”直接入栈；遇到“)”依次弹出栈中运算符入队列直到弹出“(”，若直到栈空仍未遇到“(”，则反馈非法输入；遇到运算符依次弹出栈中优先级高于或等于当前运算符的所有运算符并入队列，若遇到“(”或栈空则停止，再把当前运算符入栈。处理完所有字符后，将栈中剩余运算符弹出并入队列。
- 3、后缀表达式计算：使用队列和 double 栈。每从队列弹出一个字符进行判断，若为“0~9”或“.”则将其转化为 double 型数据入；若为运算符，则弹出栈顶两数据进行相应运算并将结果入栈，其中若遇到“/”后有“0”，反馈非法输入。队列空时弹出栈中数据作为最终结果。

二、流程图：

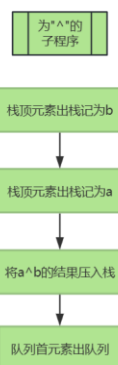
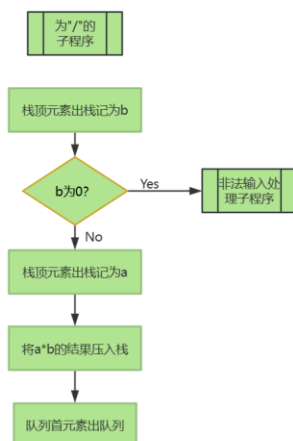
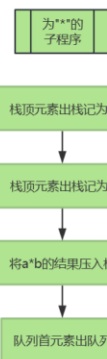
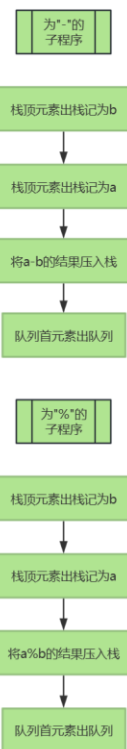
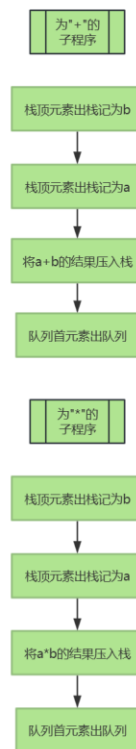
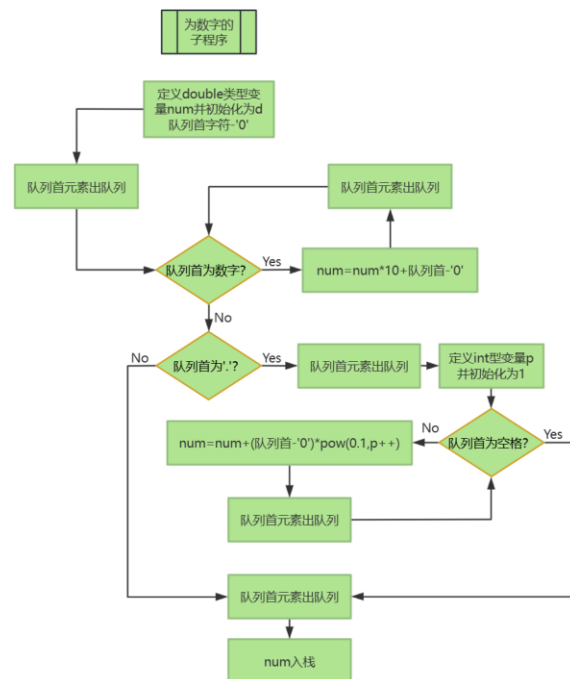
1、主要流程



2、中缀表达式转后缀表达式子程序



3、后缀表达式计算子程序



三、测试结果

		中缀表达式	测试目的
常规运算	1	$2.5+2^3/2\%3$	多个运算符号之间的混合运算
	2	$0+6-11*0$	含 0 运算
	3	$9+(3-1)*3+10/2$	带括号的运算
	4	$9.235+3.2246/2.3$	小数的运算
	5	$5+15*23+(235*2+6)*11$	大数据的计算
	6	$1/2-1$	运算结果为负数
	7	2^2^3	多个乘方运算
	8	$3+5*(5+2+3*(5/5+2/(3-2)+5)\%2)-2$	多括号运算
非法输入	9	$2*2-3+(3*5$	左括号不匹配
	10	$5.2*5.8/3)+2$	右括号不匹配
	11	$45\#5e6$	非法字符
	12	$1++2**3$	运算符并列
	13	$3/0$	除以 0
	14	$1+4(3*5)$	数字连左括号
	15	$4.+(3^5)-6$	小数点连运算符
	16	$9.(11*6)$	小数点连左括号
	17	$9*(4-6.)$	小数点连右括号
	18	$1+2..3-4$	小数点连小数点
	19	$1+.4-2$	运算符连小数点
	20	$11+8-(12-)$	运算符连右括号
	21	$12+9*(\%3+3)$	左括号连运算符
	22	$13*(.7-4*5)$	左括号连小数点
	23	$1+()-2*5$	左括号连右括号
	24	$9\%3+(12-5)8+1$	右括号连数字
	25	$7\%2/(11-9).2$	右括号连小数点
	26	$1+(12-11)(10-9)-2$	右括号连左括号

		正确输出	实际输出
常规运算	1	3.50	3.50
	2	6.00	6.00
	3	20.00	20.00
	4	10.64	10.64
	5	5586.00	5586.00
	6	-0.50	-0.50
	7	256.00	256.00
	8	36.00	36.00
非法输入	9	ERROR IN INFIX NOTATION	ERROR IN INFIX NOTATION
	10		ERROR IN INFIX NOTATION
	11		ERROR IN INFIX NOTATION
	12		ERROR IN INFIX NOTATION
	13		ERROR IN INFIX NOTATION
	14		ERROR IN INFIX NOTATION
	15		ERROR IN INFIX NOTATION
	16		ERROR IN INFIX NOTATION
	17		ERROR IN INFIX NOTATION
	18		ERROR IN INFIX NOTATION
	19		ERROR IN INFIX NOTATION
	20		ERROR IN INFIX NOTATION
	21		ERROR IN INFIX NOTATION
	22		ERROR IN INFIX NOTATION
	23		ERROR IN INFIX NOTATION
	24		ERROR IN INFIX NOTATION
	25		ERROR IN INFIX NOTATION
	26		ERROR IN INFIX NOTATION

四、分析与探讨

1、测试结果分析

本次测试基本考虑了蕴含+、-、*、/、%、^、(、)在内的全部运算符使用，由于本次测试是最终版本代码，所以并未出现什么错误。在测试中，考虑了大部分可能出现的常规运算表达式情况，并且经测试可以看出结果均正确；同时基本上考虑到了所有可能出现的非法输入，包括非法字符、括号不匹配、表达式无意义、以及运算符的非法连接，经过测试均能检测出来并反馈错误信息“ERROR IN INFIX NOTATION”

2、探讨存在的问题与改进措施

在处理中缀表达式转后缀表达式时，额外将‘/’的字符类型与其他运算符的字符类型区分开来，本意是为了处理“除以 0”的问题，但后来发现在后缀表达式计算时可以很方便的解决“除以 0”的问题，所以额外设立的‘/’字符类型并没有在代码中体现出什么特别的效果，仅仅将其与其他运算符类型一同使用，但由于其出现次数过多，修改起来困难，并且也不会对代码的时间空间复杂性有太大影响，便没有对其进行处理，这一方面有待优化。

在进行后缀表达式运算时，给每个运算符都单独编写了一段代码，这些代码大部分相同，造成了冗余，但为了代码易理解易修改，便也没有对其进行处理，这一方面有待优化。

本算法没有处理负数的计算，若想使用负数例如“-5”，可以使用“0-5”来代替，由于是在最后测试阶段才想到负数的处理，而此时代码已经较为完善，若要考虑到负数的输入，便要对本算法进行大量的修改，例如需要在检测运算符连接时，字符转换为数字时，均需要进行许多额外的修改，工

程量过大，还可能对代码造成损害，便没有实现处理负数的功能，这一方面有待优化与实现。

通过本次实验，我深入理解了中缀表达式和后缀表达式之间的转换原理，掌握了相应的算法和数据结构，并且通过编写程序实现了该功能。同时，也掌握了后缀表达式的计算方法，能够准确计算后缀表达式的值。

在实验过程中，提高了编程能力和解决问题的能力，加深了对算法和数据结构的理解。未来将继续学习和探索更多有关表达式转换和算法设计的知识，不断提升自己的能力。


```
1. #include<iostream>
2. #include<stack>
3. #include<queue>
4. #include<string>
5. #include<iomanip>
6. #include<cmath>
7.
8. using namespace std;
9.
10. void f(string x)
11. {
12.     stack<double> num; //后缀计算
13.     stack<char> sym; //中转后
14.     queue<char> exp; //后缀表达式
15.
16.     int i = 0;
17.     char temp;
18.     while (i != x.size())
19.     {
20.         int curr = -1; //当前字符类型
21.         int next = -1; //下一字符类型
22.         int next0; //下一字符
23.
24.         //判断下一字符类型
25.         if (i != x.size() - 1)
26.         {
27.
28.             next0 = x[i + 1];
29.             if (next0 >= '0' && next0 <= '9')
30.             {
31.                 next = 0;
32.             }
33.             switch (next0)
34.             {
35.                 case '.': next = 1; break;
36.                 case '/': next = 2; break;
37.                 case '(': next = 4; break;
38.                 case ')': next = 5; break;
39.                 case '+':
40.                 case '-':
41.                 case '*':
42.                 case '^':
43.                 case '%': next = 3; break;
44.             }
```

```
45. }
46.
47. // 转为后缀表达式并判断当前字符类型
48. temp = x[i++];
49. if (temp >= '0' && temp <= '9')
50. {
51.     exp.push(temp);
52.     if (next != 0 && next != 1)
53.     {
54.         exp.push(' ');
55.     }
56.     curr = 0;
57. }
58. if (temp == '.')
59. {
60.     exp.push(temp);
61.     curr = 1;
62. }
63. if (temp == '+' || temp == '-')
64. {
65.     while (!sym.empty())
66.     {
67.         if (sym.top() == '(')
68.         {
69.             break;
70.         }
71.         exp.push(sym.top());
72.         sym.pop();
73.     }
74.     sym.push(temp);
75.     curr = 3;
76. }
77. if (temp == '*' || temp == '/' || temp == '%')
78. {
79.     while (!sym.empty())
80.     {
81.         if (sym.top() == '(' || sym.top() == '+' || sym.top() == '-')
82.         {
83.             break;
84.         }
85.         exp.push(sym.top());
86.         sym.pop();
87.     }
88.     sym.push(temp);
```

```
89.     curr = 3;
90.     if (temp == '/')
91.     {
92.         curr = 2;
93.     }
94. }
95. if (temp == '^')
96. {
97.     sym.push(temp);
98.     curr = 3;
99. }
100.     if (temp == '(')
101.     {
102.         sym.push(temp);
103.         curr = 4;
104.     }
105.     if (temp == ')')
106.     {
107.         while (!sym.empty())
108.         {
109.             if (sym.top() == '(')
110.             {
111.                 break;
112.             }
113.             exp.push(sym.top());
114.             sym.pop();
115.         }
116.         if (sym.empty())
117.         {
118.             cout << "ERROR IN INFIX NOTATION" << endl;
119.             return;
120.         }
121.         sym.pop();
122.         curr = 5;
123.     }
124.
125.     //非法字符 (##@abc)
126.     if (curr == -1)
127.     {
128.         cout << "ERROR IN INFIX NOTATION" << endl;
129.         return;
130.     }
131.
132.     //比较当前与下一字符类型，检查非法输入
```

```

133.         if ((curr == 0 && next == 4) || (curr == 1 && next != 0) || ((c
            urr == 2 || curr == 3) && (next != 0 && next != 4)) || ((curr == 4) &&
            (next != 0 && next != 4)) || ((curr == 5) && ((next == 0) || (next == 1
            ) || (next == 4))))
134.     {
135.         cout << "ERROR IN INFIX NOTATION" << endl;
136.         return;
137.     }
138. }
139.
140. while (!sym.empty())
141. {
142.     if (sym.top() == '(')
143.     {
144.         cout << "ERROR IN INFIX NOTATION" << endl;
145.         return;
146.     }
147.     exp.push(sym.top());
148.     sym.pop();
149. }
150.
151. // 计算后缀表达式
152. while (!exp.empty())
153. {
154.     char cur = exp.front();
155.     if (cur >= '0' && cur <= '9')
156.     {
157.         double n = cur - '0';
158.         exp.pop();
159.         while (exp.front() >= '0' && exp.front() <= '9')
160.         {
161.             n = n * 10 + exp.front() - '0';
162.             exp.pop();
163.         }
164.         if (exp.front() == '.')
165.         {
166.             exp.pop();
167.             int p = 1;
168.             while (exp.front() != ' ')
169.             {
170.                 n = n + (exp.front() - '0') * pow(0.1, p++);
171.                 exp.pop();
172.             }
173.         }

```

```
174.         exp.pop();
175.         num.push(n);
176.         continue;
177.     }
178.     if (cur == '+')
179.     {
180.         double b = num.top();
181.         num.pop();
182.         double a = num.top();
183.         num.pop();
184.         a = a + b;
185.         num.push(a);
186.         exp.pop();
187.         continue;
188.     }
189.     if (cur == '-')
190.     {
191.         double b = num.top();
192.         num.pop();
193.         double a = num.top();
194.         num.pop();
195.         a = a - b;
196.         num.push(a);
197.         exp.pop();
198.         continue;
199.     }
200.     if (cur == '*')
201.     {
202.         double b = num.top();
203.         num.pop();
204.         double a = num.top();
205.         num.pop();
206.         a = a * b;
207.         num.push(a);
208.         exp.pop();
209.         continue;
210.     }
211.     if (cur == '/')
212.     {
213.         double b = num.top();
214.         num.pop();
215.         if (b == 0)
216.         {
217.             cout << "ERROR IN INFIX NOTATION" << endl;
```

```
218.         return;
219.     }
220.     double a = num.top();
221.     num.pop();
222.     a = a / b;
223.     num.push(a);
224.     exp.pop();
225.     continue;
226. }
227. if (cur == '%')
228. {
229.     int b = num.top();
230.     num.pop();
231.     int a = num.top();
232.     num.pop();
233.     a = a % b;
234.     num.push((double)a);
235.     exp.pop();
236.     continue;
237. }
238. if (cur == '^')
239. {
240.     double b = num.top();
241.     num.pop();
242.     double a = num.top();
243.     num.pop();
244.     a = pow(a, b);
245.     num.push(a);
246.     exp.pop();
247.     continue;
248. }
249. }
250. cout << fixed << setprecision(2) << num.top() << endl;
251. }
252.
253. int main()
254. {
255.     string IN;
256.     while (cin >> IN)
257.     {
258.         f(IN);
259.     }
260.     return 0;
261. }
```

[illegible]